Assignment: Recurrent Neural Network

Date:

Page:

Problem context:

Consider a RNN processing a sequence of three time step inputs (x_1, x_2, x_3)

The hidden state at each time step t is updated via the recurrence relation:

$$h_t = f(x_t, h_{t-1})$$

The scalar loss L is ~~eqn~~ evaluated at the final step as a function of the terminal hidden state h_3.

$$L = L(h_3)$$

1. Forward Pass

The RNN reads the sequence one step at a time. It start from an initial state h_0 (.

At each step it combines the current input with the previous hidden state to make a new hidden state:

$$h_1 = f(x_1, h_0), \quad h_2 = f(x_2, h_1), \quad h_3 = f(x_3, h_2)$$

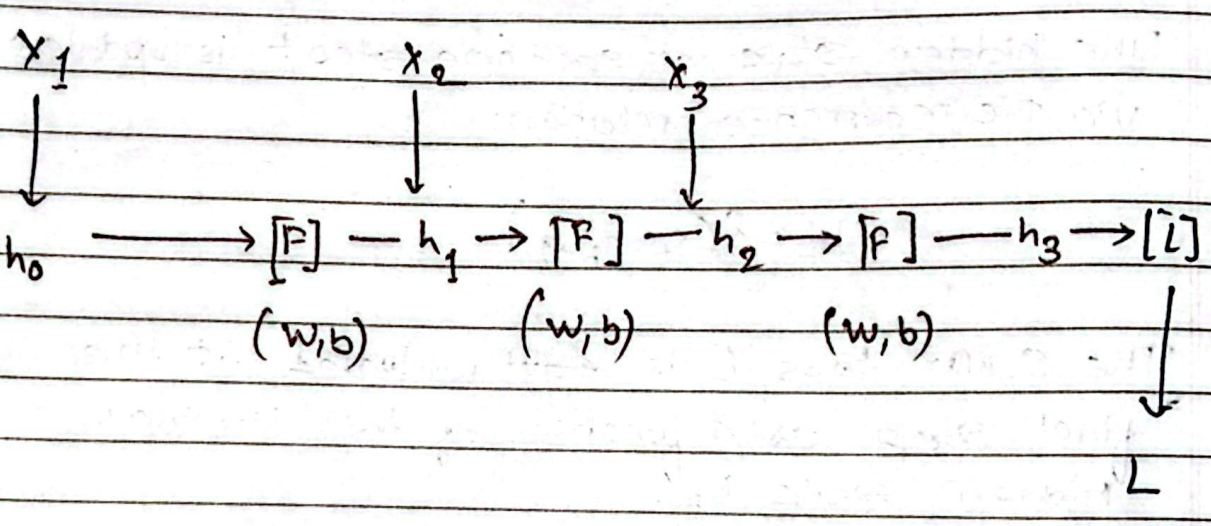
$$L = L(h_3)$$

CamScanner

Date:
Page:

A common ~~Concer~~ concrete form is $h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b)$.

Unrolled computational graph



The dependencies are

- h_1 depepends on x_1 and h_0.
- h_2 depends on x_2 and h_1.
- h_3 depends on x_3 and h_2.
- L depends only on h_3.

The important point is that every f box uses the same weights. Only the data following through it changes.

Date:
Page:

2. Hidden State Dynamics

h_t is the network's memory. It is a fixed size vector that summarises everything seen so far, in compressed form.

Context accumulations through recursion.
Unrolling the recurrence gives:

$$h_3 = f(x_3, f(x_2, f(x_1, h_0)))$$

So h_3 is a function of x_1, x_2 and x_3, even though x_1 was never fed to the network at ~~every~~ step 3. A each step the network decides how much old memory to keep (through W_{hh}) and how much new information to write in (through W_{xh}). Because state has fixed size, the network must learn what is worth remembering.

3. Back propagation Through Time

The loss L depends on h_3, which depends on h_2, which depends on h_1. Gradients therefore flow back wards along the same path!

CamScanner

Date:
Page:

$L \rightarrow h_3 \rightarrow h_2 \rightarrow h_1$ (and from each h_t to the shared weights and to x_t)

At each step, the gradient arriving at h_t is multiplied by the local jacobian $\partial h_t / \partial h_{t-1}$ to get the gradient at h_{t-1}.

It is ~~unroll~~ called through time because the network is unrolled across time steps. Each time ~~step~~ step acts like a layer of a very deep feed forward network and the backward pass travels from the last time step to the first. Ordinary backprop moves backwards through layers, BPTT moves backwards through time.

4. Chain Rule Derivation

Since L depends on h_1 only through h_2 and h_2 only through h_3

Step 1:

$$\partial L / \partial h_2 = (\partial L / \partial h_3)(\partial h_3 / \partial h_2)$$

L is a function of h_3, so dependence on h_2 goes through h_3.

Date:
Page:

Step 2: h_2 depends on h_1 so

$$\partial L/\partial h_1 = (\partial L/\partial h_2)(\partial h_2/\partial h_1)$$

Step 3: Substituting step 1 into step 2:

$$\partial L/\partial h_1 = (\partial L/\partial h_3)\cdot(\partial h_3/\partial h_2)\cdot(\partial h_2/\partial h_1)$$

As jacobian matrices with $\partial L/\partial h_3 \in R^{\wedge}(1\times h)$ and each $\partial h_t/\partial h_{t-1} \in R^{\wedge}(n\times n)$:

$$\partial L/\partial h_1 = (\partial L/\partial h_3)\, J_3\, J_2, \text{ where } J_t = \partial h_t/\partial h_{t-1}$$

For the tanh RNN, $J_t = \text{diag}(1-h_t^2)\cdot W_hh$, so:

$$\partial L/\partial h_1 = (\partial L/\partial h_3)\cdot \text{diag}(1-h_3^2)\, W_hh \cdot \text{diag}(1-h_2^2)\, W_hh$$

Written as column vector gradients each jacobian is transposed and the order reverses. ~~My code does exactly this and matches autograd~~

Date:
Page:

5. Intermediate Activations and Memory

The local Jacobians depends on the actual values of the hidden states.

For the tanh RNN, J_t needs h_t, Since $diag(1-h_t^2)$ appears in it.

The gradients of W_{xh} and W_{hh} also need h_{t-1} and x_t.

Those values only exist during the forward pass. So the frame work must store h_1, h_2, h_3 (and inputs and pre activations) until the backward pass uses them. otherwise it would have to recompute them.

In Pytorch, autograd does this automatically by recording each operation and its saved tensors in the graph. Memory cost grows linearly with sequence length

$T(O(T \cdot hidden_size \cdot batch))$.

That is why long sequences are expensive and why people use truncated BPTT or gradient checkpointing.

Date:
Page:

6. Parameter Sharing

The same W_{xh}, W_{hh} and b are used at every step for three reasons:

- Generalisation across positions: a pattern should be recognised whether it occurs early or late in the sequence.
- Variable-length sequences: the model handles any T with a fixed number of parameters.
- Efficiency: Parameter count doesn't grow with sequence length.
- Effect on gradients: because W is used at every time step, it contributes to the loss through every step. By the multivariable chain rule, its total gradient is the sum of the contributions from all time steps.

$$\partial L/\partial W = \Sigma_t (\partial L/\partial h_t)\,(\partial h_t/\partial W)\big|_t$$

So the gradients from all steps accumulate into one gradient per parameter and a single update is applied to share weights.

Date:
Page:

2. Vanishing and Exploding Gradients

For a gradient travelling from step T back to step k:

$$\partial h_T / \partial h_k = \Pi_{j=k+1}^{T} J_j$$

$$= \Pi \, \text{diag}(1 - h_j^2) \cdot W_{hh}$$

This is a product of T-k matrices. Roughly, its size behaves like $(\sigma \cdot \gamma)^{(T-k)}$, where σ is the largest singular value of W-hh and γ is the size of the activation derivative (≤ 1 for tanh).

- If $\sigma\gamma < 1$, the product shrinks exponentially and gradients vanish. Early steps receive almost no learning signal, so the network can't learn long range dependencies. Tanh makes this worse because it's derivative is at most 1 and is much smaller when saturated.

- If $\sigma\gamma > 1$, the product grows exponentially and gradients explode, causing unstable updates or NaNs.

Date:
Page:

Common remedies

- Gradient clipping for explosion.
- Gated architectures (LSTM/GRU) for vanishing.
- Careful initialisation (eg. orthogonal W_{hh}).
- Truncated BPTT.

CamScanner